Machine Learning – CSE 815

8th Semester B.Sc. Engg. Examination – 2021

University of Chittagong, Dept. of CSE

Selected Questions & Answers (Topics: KNN, SVM, Decision Tree, Neural Networks, Naive Bayes, K-Means, FAR/FRR/EER/ROC)

Contents

1 Q1(a): Machine Learning and Dimension Reduction 2

2 Q1(b): Types of Machine Learning Problems 2

3 Q1(c): Types of Machine Learning Methods – Comparison 4

4 Q2(a): FAR, FRR, EER, ROC, F-score, Sensitivity, Confusion Matrix 7

5 Q2(b): Overfitting vs Generalization 9

6 Q2(c): Cross-Validation and 4-Fold Method 9

7 Q3: Neural Network Layers, Hebb Rule, and Prototype Patterns 10

8 Q4: Naive Bayes Classifier 13

9 Q5: K-Means Clustering 14

10 Q5(b): Feature Space, Linear Separability 15

11 Q6(a): Decision Tree 16

12 Q6(b): Edit Distance 17

13 Q6(c)(d): Support Vector Machine (SVM) 18

14 Q7(a): Perceptron Convergence Proof 19

15 Q7(b): Perceptron Rule – Classification Problem 20

16 Special Topic: Hidden Markov Model (HMM) and CNN 22

17 Q8: Fruit Sorting – Perceptron, Hamming, and Hopfield Networks **25**

Q1(a): Machine Learning and Dimension Reduction

Question: What is "Machine Learning"? What is dimension reduction in Machine Learning? Give examples.

Answer:

Machine Learning (ML) is a branch of Artificial Intelligence where a system *learns patterns from data* and improves its performance on a task *without being explicitly programmed*.

$$\text{Model} = f(\text{Training Data}) \xrightarrow{\text{learns}} \text{Predictions on new data}$$

Examples:

- Spam email classification
- Face recognition
- Predicting house prices

Dimension Reduction is the process of reducing the number of input features (variables) while retaining the most important information. It helps avoid the "curse of dimensionality."

Why needed? High-dimensional data causes: (1) slow training, (2) overfitting, (3) hard visualization.

Common Methods:

- **PCA (Principal Component Analysis):** Projects data onto directions of maximum variance.
- **LDA (Linear Discriminant Analysis):** Maximizes class separability.
- **t-SNE:** Non-linear; good for visualization.

Example: A 1000-feature dataset of hand images can be reduced to 20 principal components using PCA, keeping 95% of the information (as done in hand geometry biometric systems).

Q1(b): Types of Machine Learning Problems

Question: Describe different types of Machine Learning problems with necessary examples.

Answer:

Machine Learning problems are broadly classified by the **nature of the input/output** and **what is being predicted**:

1. Classification

Predict a **discrete class label** from a fixed set of categories.

$$f : \mathcal{X} \to \{c_1, c_2, \ldots, c_K\}$$

- **Binary classification:** Two classes. Example: spam vs. not-spam email.
- **Multi-class classification:** $K > 2$ classes. Example: handwritten digit recognition (0–9).
- **Multi-label classification:** Multiple labels per instance. Example: tagging an image with "cat", "outdoor", and "sunny" simultaneously.

Algorithms: Decision Tree, SVM, KNN, Naive Bayes, Neural Networks.

2. Regression

Predict a **continuous numerical value**.

$$f : \mathcal{X} \to \mathbb{R}$$

Examples: Predicting house prices, stock market forecasting, temperature prediction.
Algorithms: Linear Regression, SVR, Neural Networks.

3. Clustering

Group data into **clusters** based on similarity, *without* predefined labels.
Examples: Customer segmentation, grouping news articles by topic, gene expression analysis.
Algorithms: K-Means, DBSCAN, Hierarchical Clustering.

4. Density Estimation

Estimate the **probability distribution** $P(X)$ of the data.
Examples: Anomaly/fraud detection (low-density points = anomalies), generative modelling.
Algorithms: Gaussian Mixture Models, KDE.

5. Dimensionality Reduction

Map high-dimensional data to a lower-dimensional representation while preserving structure.
Examples: Visualising high-dimensional gene data in 2D, compressing image features for faster classification.
Algorithms: PCA, LDA, t-SNE, Autoencoders.

6. Sequence / Structured Prediction

Predict **sequences or structured outputs** (not just a single label or number).
Examples:

- **Speech recognition:** Audio sequence → text sequence (modelled by Hidden Markov Models or RNNs).
- **Machine translation:** English sentence → Bengali sentence.
- **Part-of-speech tagging:** "The cat sat" → [DET, NOUN, VERB].

Algorithms: HMM, RNN, LSTM, Transformer.

7. Reinforcement Learning

An agent learns a **policy** (sequence of actions) to maximise cumulative reward through interaction with an environment.
Examples: Game playing (Chess, Go), robot navigation, resource scheduling.
Algorithms: Q-Learning, Policy Gradient, Actor-Critic.

Q1(c): Types of Machine Learning Methods – Comparison

Question: Describe different types of Machine Learning methods with necessary examples. Make a comparison among different machine learning methods.

Answer:

Classification of ML Methods

1. Supervised Learning

The model is trained on **labelled data** (x_i, y_i) and learns to predict y from x.

Examples and algorithms:

- **KNN:** Classifies by majority vote of k nearest neighbours. Example: face recognition.
- **SVM:** Finds maximum-margin hyperplane. Example: text categorisation.
- **Decision Tree:** Tree of feature tests. Example: medical diagnosis.
- **Naive Bayes:** Probabilistic, uses Bayes theorem. Example: spam filtering.
- **Neural Networks / CNN:** Learns hierarchical features. Example: image classification (CNN identifies edges → shapes → objects layer by layer).

2. Unsupervised Learning

The model finds **structure in unlabelled data**.

Examples and algorithms:

- **K-Means:** Groups data into K clusters. Example: customer segmentation.
- **PCA:** Reduces dimensionality by finding principal directions of variance.
- **Autoencoders:** Neural networks that learn compressed representations.
- **Hopfield / Hamming networks:** Associative memory for bipolar pattern storage and recall.

3. Semi-Supervised Learning

Uses a **small amount of labelled** data and a **large amount of unlabelled** data.
Example: Medical imaging where only a few scans are labelled by doctors.

4. Reinforcement Learning

An agent learns by **trial and error** from environmental rewards.
Example: AlphaGo learning to play the game of Go.

5. Probabilistic / Generative Methods

Model the **joint distribution** $P(X, Y)$.

- **Hidden Markov Model (HMM):** Models sequential data where the system transitions through *hidden states*. The observed outputs depend probabilistically on the hidden state. Widely used in **speech recognition** and **bioinformatics**. The model is defined by:

  $$\lambda = (A,\ B,\ \pi)$$

  where A = state transition matrix, B = emission probability matrix, π = initial state distribution.
- **Naive Bayes:** Generative classifier using conditional independence assumption.

Comparison Table

Method	Type	Training data	Interpretability	Handles noise	Best for
KNN	Supervised	Labelled	High (intuitive)	Moderate	Small datasets
SVM	Supervised	Labelled	Moderate	High (with kernel)	High-dim data
Decision Tree	Supervised	Labelled	Very high	Low (overfits)	Tabular data
Naive Bayes	Supervised	Labelled	High	High	Text classification
Neural Network	Supervised	Labelled	Low (black box)	Very high	Images, speech
CNN	Supervised	Labelled	Low	Very high	Image recognition
HMM	Generative	Labelled sequences	Moderate	High	Speech, sequences
K-Means	Unsupervised	Unlabelled	Moderate	Moderate	Clustering
Hopfield	Unsupervised	Unlabelled (bipolar)	High	Moderate	Pattern recall
Q-Learning	Reinforce.	Reward signal	Low	High	Sequential decisions

CNN (Convolutional Neural Network) – Special Highlight:

CNN is a supervised deep learning model specialised for **grid-structured data** (images, audio spectrograms). Its key layers:

- **Convolutional layer:** Applies learned filters (kernels) to detect local features (edges, textures):

$$(f * g)[i, j] = \sum_m \sum_n f[m, n] \cdot g[i - m,\ j - n]$$

- **Pooling layer:** Reduces spatial dimensions (max pooling: takes the maximum in each region).
- **Fully connected layer:** Final classification using the extracted features.

Example: LeNet-5 for digit recognition; ResNet for ImageNet classification (1000 classes, >99% accuracy).

HMM (Hidden Markov Model) – Special Highlight:

An HMM models a system that transitions through **hidden states** $q_t \in \{s_1, \ldots, s_N\}$ while producing **observable outputs** o_t:

$$P(O \mid \lambda) = \sum_Q P(O \mid Q, \lambda)\, P(Q \mid \lambda)$$

Three fundamental HMM problems:

1. **Evaluation:** Compute $P(O|\lambda)$ – Forward Algorithm.
2. **Decoding:** Find the most likely hidden state sequence – Viterbi Algorithm.
3. **Learning:** Estimate parameters λ from data – Baum-Welch (EM) Algorithm.

Example: In speech recognition, phonemes are hidden states; acoustic frames are observations. HMM decodes the spoken word from acoustic signals.

Q2(a): FAR, FRR, EER, ROC, F-score, Sensitivity, Confusion Matrix

Question: Define: (i) FAR, (ii) FRR, (iii) EER, (iv) ROC, (v) Hit Ratio, (vi) F-score, (vii) Specificity, (viii) Sensitivity, (ix) Confusion Matrix.

Answer:

Confusion Matrix (Foundation for all metrics):

	Predicted Positive	**Predicted Negative**
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

(i) False Acceptance Rate (FAR): The rate at which an impostor is *wrongly accepted* by the system.

$$\text{FAR} = \frac{\text{False Acceptances}}{\text{Total Impostor Attempts}} = \frac{FP}{FP + TN}$$

Lower FAR = more secure system.

(ii) False Rejection Rate (FRR): The rate at which a genuine user is *wrongly rejected.*

$$\text{FRR} = \frac{\text{False Rejections}}{\text{Total Genuine Attempts}} = \frac{FN}{FN + TP}$$

Lower FRR = more user-friendly system.

(iii) Equal Error Rate (EER): The point where FAR = FRR. It is a single-number summary of a biometric system's performance. *Lower EER = better system.*

$$\text{EER}: \quad \text{FAR}(t^*) = \text{FRR}(t^*)$$

(iv) Receiver Operating Characteristic (ROC): A curve that plots **Hit Rate** (TPR) vs. **FAR** (FPR) at different threshold values. The area under the ROC curve (AUC) near 1 indicates a perfect classifier.

(v) Hit Ratio (Success Rate): The proportion of genuine users correctly accepted.

$$\text{Hit Ratio} = \frac{TP}{TP + FN} = 1 - \text{FRR}$$

(vi) F-score (F1-score): Harmonic mean of Precision and Recall. Useful when classes are imbalanced.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{where Precision} = \frac{TP}{TP+FP}, \ \text{Recall} = \frac{TP}{TP+FN}$$

(vii) Specificity (True Negative Rate):

$$\text{Specificity} = \frac{TN}{TN+FP}$$

(viii) Sensitivity (Recall / TPR):

$$\text{Sensitivity} = \frac{TP}{TP+FN}$$

(ix) Confusion Matrix: A table that summarises classification performance by showing counts of correct and incorrect predictions for each class. It is the *foundation* from which FAR, FRR, Sensitivity, Specificity, and F-score are all derived.

	Predicted Positive	**Predicted Negative**	**Row Total**
Actual Positive	TP (True Positive)	FN (False Negative)	TP+FN
Actual Negative	FP (False Positive)	TN (True Negative)	FP+TN
Column Total	TP+FP	FN+TN	

Meaning of each cell:

- **TP:** Genuine user correctly accepted (correct)
- **TN:** Impostor correctly rejected (correct)
- **FP:** Impostor wrongly accepted – a security breach (wrong) → counted in FAR
- **FN:** Genuine user wrongly rejected – inconvenient (wrong) → counted in FRR

Numerical Example (biometric system, 100 genuine + 100 impostors):

	Predicted: Accept	**Predicted: Reject**
Genuine (Positive)	90 (TP)	10 (FN)
Impostor (Negative)	5 (FP)	95 (TN)

Derived metrics:

$$\text{FAR} = \frac{FP}{FP+TN} = \frac{5}{100} = 0.05, \quad \text{FRR} = \frac{FN}{FN+TP} = \frac{10}{100} = 0.10$$

$$\text{Accuracy} = \frac{TP+TN}{200} = \frac{185}{200} = 92.5\%, \quad \text{F}_1 = 2 \times \frac{0.947 \times 0.90}{0.947 + 0.90} \approx 0.923$$

Why use it? Accuracy alone is misleading on imbalanced data. The confusion matrix shows *where exactly* the model fails (FP vs FN), which matters differently depending on the application (security vs convenience).

Q2(b): Overfitting vs Generalization

Question: Discuss Overfitting vs Generalization with necessary example.

Answer:

Aspect	Overfitting	Generalization
Definition	Model memorizes training data	Model learns true patterns
Training Error	Very low	Moderate
Test Error	Very high	Low
Cause	Too complex model	Proper regularization
Analogy	Student memorizes answers	Student understands concepts

Example: A Decision Tree with unlimited depth will perfectly classify all training samples (overfitting). Pruning the tree (limiting depth) leads to better generalization on unseen data.

Fixes for Overfitting:

- More training data
- Cross-validation
- Regularization (L1/L2)
- Pruning (for trees)
- Dropout (for neural networks)

Q2(c): Cross-Validation and 4-Fold Method

Question: What is "cross-validation"? Explain 4-fold cross-validation.

Answer:

Cross-Validation is a model evaluation technique where the dataset is split into multiple subsets. The model is trained on some subsets and tested on the remaining one, repeated multiple times. This gives a reliable estimate of model performance.

4-Fold Cross-Validation:

Divide the dataset into 4 equal parts (folds):

Iteration	Fold 1	Fold 2	Fold 3	Fold 4
1	**Test**	Train	Train	Train
2	Train	**Test**	Train	Train
3	Train	Train	**Test**	Train
4	Train	Train	Train	**Test**

Final performance = Average accuracy over 4 iterations.

$$\text{Final Accuracy} = \frac{A_1 + A_2 + A_3 + A_4}{4}$$

Q3: Neural Network Layers, Hebb Rule, and Prototype Patterns

Question: (a) Define: (i) input/hidden/output layers, (ii) weights, (iii) activation functions, (iv) error.

(b) Define Hebb rule. Analyse the application of Hebb rule using an autoassociative memory.

(c) Consider the three prototype patterns p_1, p_2, p_3 (2×2 binary images) and a test pattern p_t.
(i) Use the Hebb rule to design a perceptron that recognises these three patterns p_1, p_2, p_3.
(ii) Find the response of the network to the pattern p_t. Is the response correct?

Answer:

Q3(a): Neural Network Definitions

(i) Input / Hidden / Output Layers:

- **Input Layer:** The first layer. Receives raw inputs $\mathbf{p} = [p_1, p_2, \ldots, p_R]^T$. Performs *no computation* – only distributes inputs forward.
- **Hidden Layer(s):** Intermediate layers. Each neuron computes $a = f(W\mathbf{p} + b)$. They extract non-linear features. A network can have one or many hidden layers.
- **Output Layer:** The final layer producing the prediction $\hat{y}$. Activation function chosen based on task (sigmoid, softmax, or linear).

(ii) Weights (W): Parameters on connections between neurons, determining connection strength:

$$n = \sum_{i=1}^{R} w_i p_i + b = W\mathbf{p} + b$$

Weights are *learned* by minimising error during training.

(iii) Activation Functions: Non-linear functions $a = f(n)$ applied to net input n:

Name	Formula	Typical Use
Hard Limit	$a = 1$ if $n \geq 0$, else 0	Binary output
Sign (satlins)	$a = +1$ if $n \geq 0$, else -1	Bipolar Hebb networks
Sigmoid	$a = 1/(1 + e^{-n})$	Probability output
ReLU	$a = \max(0, n)$	Deep network hidden layers
Linear	$a = n$	Regression output

(iv) Error: Difference between desired target t and actual output a:

$$e = t - a, \qquad E = \tfrac{1}{2}\sum_k e_k^2 \quad \text{(mean squared error to minimise)}$$

Q3(b): Hebb Rule and Autoassociative Memory

Hebb Rule (Donald Hebb, 1949): *"Neurons that fire together, wire together."* The connection between two neurons is strengthened when both are simultaneously active.

Mathematical form for building a weight matrix from Q training pairs (p^q, t^q):

$$W = \sum_{q=1}^{Q} t^q (p^q)^T, \qquad b = \sum_{q=1}^{Q} t^q$$

Autoassociative Memory: A special case where the *input and target are the same*: $t^q = p^q$. The network learns to store patterns and reconstruct them from partial/noisy inputs.

Weight matrix for autoassociative memory:

$$W = \sum_{q=1}^{Q} p^q (p^q)^T - Q\,I \qquad \text{(diagonal zeroed to prevent self-excitation)}$$

Worked example – store two bipolar patterns $p_1 = [+1, -1]^T$ and $p_2 = [-1, +1]^T$:

$$W = p_1 p_1^T + p_2 p_2^T - 2I = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} - \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix}$$

Recall test – exact input $p_1 = [+1, -1]^T$:

$$n = W p_1 = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \xrightarrow{\text{sign}} \begin{bmatrix} +1 \\ -1 \end{bmatrix} = p_1 \ \checkmark$$

Recall test – noisy input $\tilde{p} = [+1, +1]^T$ (bit 2 flipped):

$$n = W \tilde{p} = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix} \xrightarrow{\text{sign}} \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

The noisy input does not recover the original pattern here (patterns are not orthogonal enough). This illustrates the **limitation** of simple Hebb autoassociative memory: it works well only for orthogonal prototype patterns.

Q3(c): Perceptron for Three Prototype Patterns using Hebb Rule

Encoding the patterns:

Each 2×2 image is read row by row into a 4×1 bipolar vector (dark/black = $+1$, light/white = -1). From the exam figure:

$$p_1 = \begin{bmatrix}+1\\+1\\-1\\-1\end{bmatrix}, \quad p_2 = \begin{bmatrix}-1\\+1\\+1\\-1\end{bmatrix}, \quad p_3 = \begin{bmatrix}-1\\-1\\+1\\+1\end{bmatrix}, \quad p_t = \begin{bmatrix}+1\\-1\\+1\\-1\end{bmatrix} \text{ (test)}$$

(i) Design using Hebb Rule:

Assign targets: $t_1 = +1$, $t_2 = +1$, $t_3 = -1$.

$$W = t_1 p_1^T + t_2 p_2^T + t_3 p_3^T$$
$$= (+1)[+1, +1, -1, -1] + (+1)[-1, +1, +1, -1] + (-1)[-1, -1, +1, +1]$$

Computing column by column:

	w_1	w_2	w_3	w_4
From p_1 $(t = +1)$	$+1$	$+1$	-1	-1
From p_2 $(t = +1)$	-1	$+1$	$+1$	-1
From p_3 $(t = -1)$	$+1$	$+1$	-1	-1
Sum $= W$	$\mathbf{+1}$	$\mathbf{+3}$	$\mathbf{-1}$	$\mathbf{-3}$

$$\boxed{W = [+1,\ +3,\ -1,\ -3]}, \qquad b = t_1 + t_2 + t_3 = 1 + 1 - 1 = \boxed{1}$$

Verify on all three training patterns:

Pattern	$n = Wp + b$	$a = \text{sign}(n)$	**Ta**
p_1	$(1)(1) + (3)(1) + (-1)(-1) + (-3)(-1) + 1 = 1 + 3 + 1 + 3 + 1 = 9$	$+1$	$+$
p_2	$(1)(-1) + (3)(1) + (-1)(1) + (-3)(-1) + 1 = -1 + 3 - 1 + 3 + 1 = 5$	$+1$	$+$
p_3	$(1)(-1) + (3)(-1) + (-1)(1) + (-3)(1) + 1 = -1 - 3 - 1 - 3 + 1 = -7$	-1	$-$

All training patterns are correctly classified. ✓

(ii) Response to Test Pattern $p_t = [+1, -1, +1, -1]^T$:

$$n = Wp_t + b = (1)(1) + (3)(-1) + (-1)(1) + (-3)(-1) + 1 = 1 - 3 - 1 + 3 + 1 = 1$$

$$a = \text{sign}(1) = \mathbf{+1}$$

The network outputs $+1$, classifying p_t in the same class as p_1 and p_2.

Is the response correct? Check Hamming distances:

$$d_H(p_t, p_1) = |\{-1 \neq +1,\ -1 \neq +1\}| = 2 \quad \text{(positions 2 and 4 differ)}$$

$$d_H(p_t, p_2) = |\{+1 \neq -1,\ +1 \neq -1\}| = 2 \quad \text{(positions 1 and 3 differ)}$$

$$d_H(p_t, p_3) = |\{+1 \neq -1,\ +1 \neq -1\}| = 2 \quad \text{(positions 1 and 3 differ)}$$

p_t is **equidistant** from all three prototypes (Hamming distance = 2 in all cases). The perceptron's answer of +1 (favouring p_1/p_2) is **not uniquely correct** since the pattern is ambiguous. This shows a key limitation: when an input is equidistant from multiple prototypes, no classifier can give a definitive answer. A Hamming network would show equal scores for all three prototypes, confirming the ambiguity.

Q4: Naive Bayes Classifier

Question: (a) How would you apply Naive Bayes to categorical and numerical features?
(b) Does Naive Bayes fit the generative or discriminative category?
(c) Classify test sample (A=Low, B=Fair, C=Low) using Naive Bayes on given dataset.

Answer:

(a) Naive Bayes for Different Feature Types:

- **Categorical features:** Use *Multinomial* or *Bernoulli* Naive Bayes. Compute $P(\text{feature value} \mid \text{class})$ from frequency counts.
- **Numerical features:** Use *Gaussian* Naive Bayes. Assume each feature follows a normal distribution:

$$P(x \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right)$$

(b) Generative vs Discriminative:

Naive Bayes is a **Generative** classifier. It models the joint probability $P(X, Y) = P(X \mid Y) \cdot P(Y)$ and uses Bayes' theorem to compute $P(Y \mid X)$. Discriminative classifiers (like SVM, Logistic Regression) directly model $P(Y \mid X)$.

(c) Classification Example:

Dataset:

Age (A)	Income (B)	Credit (C)	Loan (Class)
High	Fair	Low	60,000
Low	Fair	High	70,000
Low	Excellent	High	80,000
High	Fair	High	65,000

Let's classify into two groups: Low Loan ($\leq$65,000) and High Loan (>65,000).

Classes: Low = {60000, 65000} (2 instances), High = {70000, 80000} (2 instances).

$P(\text{Low}) = \frac{2}{4} = 0.5, \quad P(\text{High}) = \frac{2}{4} = 0.5$

Test Sample: A = Low, B = Fair, C = Low

Using Laplacian correction (+1 to each count, denominator + number of values):

For **Low class**: A=Low (1/2), B=Fair (2/2), C=Low (1/2)

$P(\text{Low}) \times P(A{=}\text{Low}|\text{Low}) \times P(B{=}\text{Fair}|\text{Low}) \times P(C{=}\text{Low}|\text{Low}) = 0.5 \times 0.5 \times 1.0 \times 0.5 = 0.125$

For **High class**: A=Low (1/2), B=Fair (1/2), C=Low (0/2 → use Laplace: 1/4)

$P(\text{High}) \times P(A{=}\text{Low}|\text{High}) \times P(B{=}\text{Fair}|\text{High}) \times P(C{=}\text{Low}|\text{High}) = 0.5 \times 0.5 \times 0.5 \times 0.25 = 0.03125$

Result: $0.125 > 0.03125 \Rightarrow$ Predicted class = **Low Loan**.

Q5: K-Means Clustering

Question: (a) Can K-Means clustering stabilize during further rounds?
(c) Apply K-Means to CGPA data: A=3.45, B=2.78, C=2.90, D=3.53, E=2.25, F=3.67 with initial centers A and C.

Answer:

(a) Yes! K-Means converges when cluster assignments no longer change between iterations. This happens when each point is already closest to its current cluster center. The algorithm is guaranteed to converge (though possibly to a local optimum).

(c) K-Means Solution:

Initial Centers: $\mu_1 = A = 3.45$, $\mu_2 = C = 2.90$

Iteration 1 – Assign each point to nearest center:

Student	CGPA	$\lvert x - \mu_1 \rvert$	$\lvert x - \mu_2 \rvert$	Cluster
A	3.45	0.00	0.55	Cluster 1
B	2.78	0.67	0.12	Cluster 2
C	2.90	0.55	0.00	Cluster 2
D	3.53	0.08	0.63	Cluster 1
E	2.25	1.20	0.65	Cluster 2
F	3.67	0.22	0.77	Cluster 1

Update Centers:

$$\mu_1 = \frac{3.45 + 3.53 + 3.67}{3} = \frac{10.65}{3} = 3.55$$

$$\mu_2 = \frac{2.78 + 2.90 + 2.25}{3} = \frac{7.93}{3} = 2.643$$

Iteration 2 – Re-assign with new centers:

Student	CGPA	$\lvert x - 3.55\rvert$	$\lvert x - 2.643\rvert$	Cluster
A	3.45	0.10	0.807	Cluster 1
B	2.78	0.77	0.137	Cluster 2
C	2.90	0.65	0.257	Cluster 2
D	3.53	0.02	0.887	Cluster 1
E	2.25	1.30	0.393	Cluster 2
F	3.67	0.12	1.027	Cluster 1

Same assignments as before $\Rightarrow$ **Converged!**

Final Clusters:

- **Cluster 1 (Higher CGPA):** A (3.45), D (3.53), F (3.67)
- **Cluster 2 (Lower CGPA):** B (2.78), C (2.90), E (2.25)

Q5(b): Feature Space, Linear Separability

Question: What do you understand by feature space? What is linearly separable and not separable? Explain with example.

Answer:

Feature Space

A **feature space** is an n-dimensional mathematical space where each axis represents one feature (attribute) of the data. Each data point is a vector in this space:

$$x = [x_1,\ x_2,\ \ldots,\ x_n]^T \in \mathbb{R}^n$$

Example: If we describe a fruit by two features – weight (x_1) and diameter (x_2) – the feature space is $\mathbb{R}^2$. Each fruit is a point (x_1, x_2) in this 2D space.

Why it matters: The geometry of the feature space determines which classifiers can be applied. In a biometric system (e.g. hand geometry), each hand is a point in a high-dimensional feature space (finger lengths, widths, palm area, etc.).

Linearly Separable

Two classes are **linearly separable** if a single **hyperplane** (line in 2D, plane in 3D, hyperplane in nD) can perfectly separate them:

$$\text{Class A} : W \cdot x + b > 0 \qquad \text{Class B} : W \cdot x + b < 0$$

Example (2D):

Points: Class A $= \{(1,1),\ (2,1),\ (1,2)\}$; Class B $= \{(-1,-1),\ (-2,-1),\ (-1,-2)\}$.

These can be separated by the line $x_1 + x_2 = 0$. A simple perceptron or SVM with a linear kernel can solve this.

$$\text{Decision boundary (line):} \quad x_1 + x_2 = 0$$

Key property: A **single-layer perceptron** can solve *only* linearly separable problems. SVM with a linear kernel also requires linear separability (or uses soft margin for approximate separation).

Not Linearly Separable

Two classes are **not linearly separable** if no single hyperplane can separate them without errors.

Classic example – XOR problem:

x_1	x_2	Class
0	0	0 (Class B)
0	1	1 (Class A)
1	0	1 (Class A)
1	1	0 (Class B)

Class A points $(0, 1)$ and $(1, 0)$ and Class B points $(0, 0)$ and $(1, 1)$ cannot be separated by any single line in 2D – XOR is not linearly separable.

Solutions for non-linear problems:

- **Kernel trick (SVM):** Map data to a higher-dimensional space where it becomes linearly separable:

$$\phi : \mathbb{R}^n \to \mathbb{R}^m \ (m \gg n), \quad K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

- **Multi-layer neural networks (MLP/CNN):** Hidden layers learn non-linear feature transformations, creating curved decision boundaries.
- **Decision Trees:** Create axis-aligned rectangular boundaries.

Summary:

Property	**Linearly Separable**	**Not Linearly Separable**
Boundary shape	Straight hyperplane	Curve / complex boundary
Example	AND, OR gates	XOR gate
Solved by	Perceptron, Linear SVM	MLP, RBF-SVM, CNN, Tree

Q6(a): Decision Tree

Question: Write short notes on Decision Tree.

Answer:

A **Decision Tree** is a tree-shaped model where:

- **Internal nodes** represent feature tests (e.g., Age $>$ 30?)
- **Branches** represent outcomes of tests
- **Leaf nodes** represent class labels or output values

How it works:

At each node, the best feature is selected using an impurity measure:

$$\text{Entropy}(S) = -\sum_i p_i \log_2 p_i$$

$$\text{Gini Index} = 1 - \sum_i p_i^2$$

$$\text{Information Gain} = \text{Entropy}(S) - \sum_v \frac{|S_v|}{|S|}\text{Entropy}(S_v)$$

The feature with the **highest information gain** is chosen to split.

Advantages: Easy to interpret, no feature scaling needed, handles both numerical and categorical data.

Disadvantages: Prone to overfitting; pruning is needed.

Q6(b): Edit Distance

Question: What is Edit Distance? Write down the algorithm. Find the Edit Distance between 'exercised' and 'executed'.

Answer:

Edit Distance (Levenshtein Distance) is the minimum number of single-character operations (insert, delete, substitute) needed to transform one string into another.

Algorithm (Dynamic Programming):

Let s_1 = "exercised", s_2 = "executed". Build a matrix D where:

$$D[i][j] = \begin{cases} D[i-1][j-1] & \text{if } s_1[i] = s_2[j] \quad \text{(no cost)} \\ 1 + \min(D[i-1][j],\ D[i][j-1],\ D[i-1][j-1]) & \text{otherwise} \end{cases}$$

Computation:

s_1 = e-x-e-r-c-i-s-e-d (length 9), s_2 = e-x-e-c-u-t-e-d (length 8)

	ε	e	x	e	r	c	i	s	e	d
ε	0	1	2	3	4	5	6	7	8	9
e	1	0	1	2	3	4	5	6	7	8
x	2	1	0	1	2	3	4	5	6	7
e	3	2	1	0	1	2	3	4	5	6
c	4	3	2	1	1	1	2	3	4	5
u	5	4	3	2	2	2	2	3	4	5
t	6	5	4	3	3	3	3	3	4	5
e	7	6	5	4	4	4	4	4	3	4
d	8	7	6	5	5	5	5	5	4	**3**

Edit Distance = 3 (3 operations needed: replace 'r' with 'c', replace 'i' with 'u', replace 's' with 't')

Q6(c)(d): Support Vector Machine (SVM)

Question: (c) What is a support vector? Why maximum margin in SVM?
(d) What is the "kernel trick"? How do you compute M in terms of W and b?

Answer:

(c) Support Vectors and Maximum Margin:

Support Vectors are the data points *closest* to the decision boundary (hyperplane). They "support" or define the position and orientation of the hyperplane.

The **decision hyperplane** in SVM is:

$$W \cdot x + b = 0$$

The **margin** is the total distance between the two parallel supporting hyperplanes:

$$W \cdot x + b = +1 \quad \text{(positive support hyperplane)}$$

$$W \cdot x + b = -1 \quad \text{(negative support hyperplane)}$$

$$\text{Margin} = M = \frac{2}{\|W\|}$$

Why maximize margin? A larger margin gives better generalization. It reduces the chance of misclassification on unseen data. SVM finds the hyperplane that **maximizes** this margin by solving:

$$\min \frac{1}{2}\|W\|^2 \quad \text{subject to } y_i(W \cdot x_i + b) \geq 1$$

(d) Kernel Trick:

When data is **not linearly separable**, we map it to a higher-dimensional space using a function $\phi(x)$. The kernel trick avoids computing $\phi(x)$ explicitly by using a **kernel function**:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

Common Kernels:

- **Linear:** $K(x_i, x_j) = x_i \cdot x_j$
- **Polynomial:** $K(x_i, x_j) = (x_i \cdot x_j + c)^d$
- **RBF (Gaussian):** $K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

Computing M in terms of W and b:

The margin M is the perpendicular distance between the two supporting hyperplanes:

$$M = \frac{2}{\|W\|}$$

The distance from origin to the hyperplane $W \cdot x + b = 0$ is $\frac{|b|}{\|W\|}$.

Q7(a): Perceptron Convergence Proof

Question: Demonstrate a proof of convergence for the perceptron learning rule, for a single neuron perceptron.

Answer:

Setup:

Consider a single-neuron perceptron with weight vector $W \in \mathbb{R}^R$ and no bias (absorbed into W by augmenting the input). The perceptron update rule is:

$$W(t+1) = W(t) + e(t) \cdot p(t), \qquad e(t) = t(t) - a(t)$$

where $a(t) = \text{hardlim}(W(t) \cdot p(t))$ and targets are bipolar: $t \in \{+1, -1\}$.

Theorem (Perceptron Convergence Theorem):

If the training data is **linearly separable**, the perceptron learning rule will **converge in a finite number of steps**.

Proof:

Step 1 – Assume linear separability.

Since the data is linearly separable, there exists an optimal weight vector W^* and a margin $\delta > 0$ such that for all training pairs (p^q, t^q):

$$t^q(W^* \cdot p^q) \geq \delta > 0 \quad \text{for all } q$$

Without loss of generality, normalise: $\|W^*\| = 1$.

Step 2 – Lower bound on $W(t) \cdot W^*$.

Each update occurs when there is an error: $t \cdot (W \cdot p) \leq 0$.

After k errors, starting from $W(0) = 0$:

$$W(k) \cdot W^* = W(k-1) \cdot W^* + t^q(W^* \cdot p^q) \geq W(k-1) \cdot W^* + \delta$$

By induction over k error steps:

$$\boxed{W(k) \cdot W^* \geq k\delta}$$

Step 3 – Upper bound on $\|W(k)\|^2$.

Let $M = \max_q \|p^q\|^2$. At each error step:

$$\|W(k)\|^2 = \|W(k-1) + t^q p^q\|^2 = \|W(k-1)\|^2 + 2\,t^q\,(W(k-1) \cdot p^q) + \|p^q\|^2$$

Since an error means $t^q(W(k-1) \cdot p^q) \leq 0$, the middle term is ≤ 0:

$$\|W(k)\|^2 \leq \|W(k-1)\|^2 + \|p^q\|^2 \leq \|W(k-1)\|^2 + M$$

By induction:

$$\boxed{\|W(k)\|^2 \leq kM}$$

Step 4 – Combine bounds using Cauchy-Schwarz.

By the Cauchy-Schwarz inequality:

$$W(k) \cdot W^* \leq \|W(k)\| \cdot \|W^*\| = \|W(k)\|$$

From Steps 2 and 3:

$$k\delta \leq W(k) \cdot W^* \leq \|W(k)\| \leq \sqrt{kM}$$

Therefore:

$$k\delta \leq \sqrt{kM} \implies k^2\delta^2 \leq kM \implies k \leq \frac{M}{\delta^2}$$

Conclusion:

The number of error-corrections k is **bounded above** by $\frac{M}{\delta^2}$:

$$\boxed{k \leq \frac{\max_q \|p^q\|^2}{\delta^2}}$$

Since k is finite, the perceptron makes *at most* a finite number of weight updates and **must converge** to a solution in finite steps. ■

Physical interpretation:

- δ = margin of separation. Larger margin ⇒ faster convergence (fewer corrections needed).
- M = maximum input magnitude. Larger inputs ⇒ slower convergence.
- If the data is *not* linearly separable, $\delta = 0$ and the bound becomes infinite – the perceptron never converges (cycles forever).

Q7(b): Perceptron Rule – Classification Problem

Question: Solve the classification problem with perceptron rule. Input vectors and targets given. Use $W(0) = [0\ 0]$, $b(0) = 0$.

$$\left\{p_1 = \begin{bmatrix}2\\2\end{bmatrix}, t_1 = 0\right\} \left\{p_2 = \begin{bmatrix}1\\-2\end{bmatrix}, t_2 = 1\right\} \left\{p_3 = \begin{bmatrix}-2\\2\end{bmatrix}, t_3 = 0\right\} \left\{p_4 = \begin{bmatrix}-1\\1\end{bmatrix}, t_4 = 1\right\}$$

Answer:

Perceptron Update Rule:

$$W_{\text{new}} = W_{\text{old}} + (t - a) \cdot p^T, \quad b_{\text{new}} = b_{\text{old}} + (t - a)$$

where $a = \text{hardlim}(W \cdot p + b)$:

$$a = \begin{cases} 1 & \text{if } Wp + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Initial: $W = [0\ 0]$, $b = 0$

Pass 1:

p_1: $n = [0\ 0]\begin{bmatrix}2\\2\end{bmatrix} + 0 = 0$, $a = 1$, $t_1 = 0$, error $= 0 - 1 = -1$

$W = [0\ 0] + (-1)[2\ 2] = [-2\ \ -2]$, $b = 0 + (-1) = -1$

p_2: $n = [-2\ \ -2]\begin{bmatrix}1\\-2\end{bmatrix} + (-1) = -2 + 4 - 1 = 1$, $a = 1$, $t_2 = 1$, error $= 0 \Rightarrow$ No update.

p_3: $n = [-2\ \ -2]\begin{bmatrix}-2\\2\end{bmatrix} + (-1) = 4 - 4 - 1 = -1$, $a = 0$, $t_3 = 0$, error $= 0 \Rightarrow$ No update.

p_4: $n = [-2\ \ -2]\begin{bmatrix}-1\\1\end{bmatrix} + (-1) = 2 - 2 - 1 = -1$, $a = 0$, $t_4 = 1$, error $= 1$

$W = [-2\ \ -2] + (1)[-1\ 1] = [-3\ \ -1]$, $b = -1 + 1 = 0$

Pass 2:

p_1: $n = [-3\ \ -1]\begin{bmatrix}2\\2\end{bmatrix} + 0 = -6 - 2 = -8$, $a = 0$, $t_1 = 0 \Rightarrow$ No update.

p_2: $n = [-3\ \ -1]\begin{bmatrix}1\\-2\end{bmatrix} + 0 = -3 + 2 = -1$, $a = 0$, $t_2 = 1$, error $= 1$

$W = [-3\ \ -1] + (1)[1\ \ -2] = [-2\ \ -3]$, $b = 0 + 1 = 1$

p_3: $n = [-2\ \ -3]\begin{bmatrix}-2\\2\end{bmatrix} + 1 = 4 - 6 + 1 = -1$, $a = 0$, $t_3 = 0 \Rightarrow$ No update.

p_4: $n = [-2\ \ -3]\begin{bmatrix}-1\\1\end{bmatrix} + 1 = 2 - 3 + 1 = 0$, $a = 1$, $t_4 = 1 \Rightarrow$ No update.

Pass 3 – Verify all points:

p_1: $n = [-2\ \ -3][2\ 2]^T + 1 = -4 - 6 + 1 = -9 < 0$, $a = 0$, $t = 0$ ✓

p_2: $n = [-2\ \ -3][1\ \ -2]^T + 1 = -2 + 6 + 1 = 5 \geq 0$, $a = 1$, $t = 1$ ✓

p_3: $n = [-2\ \ -3][-2\ 2]^T + 1 = 4 - 6 + 1 = -1 < 0$, $a = 0$, $t = 0$ ✓

p_4: $n = [-2\ \ -3][-1\ 1]^T + 1 = 2 - 3 + 1 = 0 \geq 0$, $a = 1$, $t = 1$ ✓

Final solution: $W = [-2\ \ -3]$, $b = 1$

Decision boundary: $-2x_1 - 3x_2 + 1 = 0$

Special Topic: Hidden Markov Model (HMM) and CNN

Hidden Markov Model (HMM)

Question: What is a Hidden Markov Model? How is it used in Machine Learning? Describe its key algorithms.

Answer:

Definition:

A **Hidden Markov Model (HMM)** is a statistical model for systems that evolve through a sequence of *hidden* (unobservable) states, each producing an *observable* output. It is a generative probabilistic model for sequences.

Formal definition: An HMM λ is a 5-tuple:

$$\lambda = (N,\ M,\ A,\ B,\ \pi)$$

- N = number of hidden states $\{s_1, s_2, \ldots, s_N\}$
- M = number of observation symbols $\{v_1, v_2, \ldots, v_M\}$
- $A = [a_{ij}]$ = **state transition matrix:** $a_{ij} = P(q_{t+1} = s_j \mid q_t = s_i)$
- $B = [b_j(k)]$ = **emission (observation) probability matrix:** $b_j(k) = P(o_t = v_k \mid q_t = s_j)$
- $\pi = [\pi_i]$ = **initial state distribution:** $\pi_i = P(q_1 = s_i)$

Key Markov assumptions:

$$P(q_t \mid q_{t-1}, q_{t-2}, \ldots) = P(q_t \mid q_{t-1}) \quad \text{(first-order Markov)}$$

$$P(o_t \mid q_t, q_{t-1}, \ldots) = P(o_t \mid q_t) \quad \text{(output independence)}$$

Example – Speech Recognition:

HMM component	Speech recognition mapping
Hidden states q_t	Phonemes ("ah", "eh", "sh", ...)
Observations o_t	Acoustic feature vectors (MFCC frames)
Transition A	Probability of phoneme i followed by phoneme j
Emission B	Probability of acoustic frame given a phoneme

Three fundamental HMM problems and their algorithms:

Problem	Goal	Algorithm
1. Evaluation	Compute $P(O \mid \lambda)$: probability of an observation sequence given the model	**Forward Algorithm** (dynamic programming)
2. Decoding	Find the most likely hidden state sequence $Q^* = \arg\max_Q P(Q \mid O, \lambda)$	**Viterbi Algorithm**
3. Learning	Estimate parameters $\hat{\lambda} = \arg\max_\lambda P(O \mid \lambda)$ from training data	**Baum-Welch Algorithm** (EM)

Forward Algorithm (Problem 1):

Define $\alpha_t(i) = P(o_1, o_2, \ldots, o_t, q_t = s_i \mid \lambda)$. Then:

$$\alpha_1(i) = \pi_i \cdot b_i(o_1)$$

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i)\, a_{ij}\right] b_j(o_{t+1})$$

$$P(O \mid \lambda) = \sum_{i=1}^{N} \alpha_T(i)$$

Applications of HMM:

- **Speech recognition:** Each word/phoneme is modelled as an HMM; decoding finds the spoken word from audio.
- **Handwriting recognition:** Stroke sequences modelled as hidden states.
- **Bioinformatics:** Gene finding in DNA sequences (hidden states = coding/non-coding regions).
- **Finance:** Modelling regime changes in financial markets.
- **Gesture recognition:** Hand gesture sequences classified using HMMs.

Convolutional Neural Network (CNN)

Question: Describe Convolutional Neural Networks (CNN) as a Machine Learning method. What makes CNN suited for image recognition tasks?

Answer:

Definition:

A **Convolutional Neural Network (CNN)** is a deep supervised learning architecture designed to process data with a **grid topology** (images, audio spectrograms, video). It uses *shared weights* (convolutional filters) to detect local patterns, drastically reducing the number of parameters compared to a fully connected network.

Architecture (typical layers):

Input Image $\rightarrow$ [Conv $\rightarrow$ Activation $\rightarrow$ Pool]$^{\times L}$ $\rightarrow$ Flatten $\rightarrow$ FC layers $\rightarrow$ Softmax Output

1. Convolutional Layer:

Applies K learned filters of size $F \times F$ to the input to produce feature maps:

$$S[i,j] = (I * F)[i,j] = \sum_m \sum_n I[i+m,\ j+n] \cdot F[m,n] + b$$

- Early filters detect **low-level features**: edges, corners, colours.
- Deeper filters detect **high-level features**: eyes, wheels, faces.

2. Activation Function:

ReLU is standard: $a = \max(0, n)$. Introduces non-linearity and sparsity.

3. Pooling Layer:

Reduces spatial dimensions (downsampling):

$$\text{Max Pooling: } y[i,j] = \max_{(m,n)\in \text{region}} x[m,n]$$

Provides **translation invariance** – the network recognises a cat whether it is in the top-left or bottom-right of the image.

4. Fully Connected Layer:

Flattened feature maps passed through dense layers for final classification.

5. Output Layer (Softmax):

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Why CNN outperforms standard MLP for images:

Property	Standard MLP	CNN
Weight sharing	No (each pixel gets unique weights)	Yes (same filter slides over imag
Parameters	Very large ($224 \times 224 \times 3 \approx 150k$ inputs)	Compact (filter size e.g. 3×3)
Translation invariance	No	Yes (via pooling)
Local feature detection	No	Yes (filters detect local patterns

Training CNN (Backpropagation through convolutions):

Loss function (cross-entropy):

$$\mathcal{L} = -\sum_{k} y_k \log \hat{y}_k$$

Gradients flow back through fully connected → pooling → convolutional layers via chain rule.

Famous CNN architectures:

Architecture	Year	Layers	Achievement
LeNet-5	1998	7	Handwritten digit recognition
AlexNet	2012	8	Won ImageNet (top-5 error: 15.3%)
VGGNet	2014	19	Simple 3×3 filters, very deep
ResNet-50	2015	50	Skip connections; 3.57% ImageNet error

Applications of CNN:

- Image classification (ImageNet, medical imaging)
- Object detection (YOLO, Faster R-CNN)
- Face recognition and biometric verification
- Hand gesture recognition
- Natural language processing (1D convolutions on text)

CNN vs. HMM – Comparison:

Aspect	CNN	HMM
Data type	Grid data (images, spectrograms)	Sequential data (speech, genes)
Learning	Discriminative (supervised, backprop)	Generative (EM / Baum-Welch)
Interpretability	Low (black box)	Moderate (states have meaning)
Sequence modelling	Poor (needs RNN/LSTM)	Excellent
Image classification	Excellent	Poor
Training data needed	Large	Moderate

Q8: Fruit Sorting – Perceptron, Hamming, and Hopfield Networks

Question: A produce dealer has a warehouse storing fruits and vegetables. When fruit is brought to the warehouse, various types may be mixed together. A conveyer belt passes fruit through sensors measuring three properties: *shape*, *texture*, and *weight*.

Sensor encoding rules:

- Shape sensor: $+1$ if approximately round, -1 if more elliptical.
- Texture sensor: $+1$ if the surface is smooth, -1 if it is rough.
- Weight sensor: $+1$ if the fruit is more than one pound, -1 if less than one pound.

The three sensor outputs form a three-dimensional input vector: $p = [\text{shape, texture, weight}]^T$. The network must decide whether the fruit is an **orange** (p_1) or an **apple** (p_2).

(a) Write down the prototype templates for orange (p_1) and apple (p_2).
(b) Design a perceptron to recognise these patterns.
(c) Design a Hamming network to recognise these patterns.
(d) Design a Hopfield network to recognise these patterns.
(e) Test the operation of your networks by applying several different input patterns. Discuss the advantages and disadvantages of each network.

Answer:

(a) Prototype Templates

Orange (p_1):

- Shape: round $\rightarrow +1$
- Texture: rough (orange peel is rough) $\rightarrow -1$
- Weight: less than one pound $\rightarrow -1$

$$p_1 = \begin{bmatrix} +1 \\ -1 \\ -1 \end{bmatrix} \quad \text{(Orange)}$$

Apple (p_2):

- Shape: round $\rightarrow +1$
- Texture: smooth (apple skin is smooth) $\rightarrow +1$
- Weight: less than one pound $\rightarrow -1$

$$p_2 = \begin{bmatrix} +1 \\ +1 \\ -1 \end{bmatrix} \quad \text{(Apple)}$$

Key observation: Both fruits are round and light. They differ *only* in the **texture** dimension. Any network must rely entirely on texture to distinguish them.

(b) Perceptron Design

Targets (bipolar): $t_1 = +1$ (Orange), $t_2 = -1$ (Apple).

Step 1 – Compute weight vector using Hebb rule:

$$W = t_1 p_1^T + t_2 p_2^T = (+1)[+1,\ -1,\ -1] + (-1)[+1,\ +1,\ -1]$$

$$= [+1,\ -1,\ -1] + [-1,\ -1,\ +1] = [0,\ -2,\ 0]$$

$$b = t_1 + t_2 = (+1) + (-1) = 0$$

Perceptron: $W = [0 \quad -2 \quad 0]$, $b = 0$, activation $a = \text{sign}(n)$.

Notice $w_1 = 0$ and $w_3 = 0$: the network correctly ignores shape and weight (which are identical for both fruits) and relies *only* on $w_2 = -2$ (texture).

Step 2 – Verify on training prototypes:

Test orange $p_1 = [+1, -1, -1]^T$*:*

$$n = [0\ -2\ 0] \begin{bmatrix} +1 \\ -1 \\ -1 \end{bmatrix} + 0 = (0)(1) + (-2)(-1) + (0)(-1) = 2 \implies a = \text{sign}(2) = +1 = t_1 \checkmark \textbf{ Orange}$$

Test apple $p_2 = [+1, +1, -1]^T$*:*

$$n = [0\ -2\ 0] \begin{bmatrix} +1 \\ +1 \\ -1 \end{bmatrix} + 0 = (0)(1) + (-2)(+1) + (0)(-1) = -2 \implies a = \text{sign}(-2) = -1 = t_2 \checkmark \textbf{ Apple}$$

Both prototypes correctly classified. The decision rule is simply: *if texture is rough* (-1) *classify as orange; if smooth* $(+1)$ *classify as apple.*

(c) Hamming Network Design

The Hamming network finds the prototype with minimum Hamming distance from the input using two layers ($R = 3$ inputs, $S = 2$ prototype neurons).

Layer 1 (Feedforward layer) – parameters set from prototypes:

$$W^1 = \frac{1}{2} \begin{bmatrix} p_1^T \\ p_2^T \end{bmatrix} = \frac{1}{2} \begin{bmatrix} +1 & -1 & -1 \\ +1 & +1 & -1 \end{bmatrix} = \begin{bmatrix} +0.5 & -0.5 & -0.5 \\ +0.5 & +0.5 & -0.5 \end{bmatrix}$$

$$b^1 = \frac{R}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{3}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}$$

Layer 1 output: $a^1 = W^1 p + b^1$. The value $a_i^1 = R - d_H(p,\ p_i)$, so *higher score = closer prototype.*

Layer 2 (MAXNET – competitive layer): Iteratively suppresses all but the largest activation. The winning neuron identifies the recognised class.

Test orange $p_1 = [+1, -1, -1]^T$**:**

$$a^1 = \begin{bmatrix} 0.5 & -0.5 & -0.5 \\ 0.5 & 0.5 & -0.5 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 0.5 + 0.5 + 0.5 \\ 0.5 - 0.5 + 0.5 \end{bmatrix} + \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

Neuron 1 wins ($3 > 2$). Hamming distances: $d_H(p_1, p_1) = 0$, $d_H(p_1, p_2) = 1$. $\Rightarrow$ **Orange** ✓

Test apple $p_2 = [+1, +1, -1]^T$**:**

$$a^1 = \begin{bmatrix} 0.5 & -0.5 & -0.5 \\ 0.5 & 0.5 & -0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 0.5 - 0.5 + 0.5 \\ 0.5 + 0.5 + 0.5 \end{bmatrix} + \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix} + \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Neuron 2 wins ($3 > 2$). Hamming distances: $d_H(p_2, p_1) = 1$, $d_H(p_2, p_2) = 0$. $\Rightarrow$ **Apple** ✓

(d) Hopfield Network Design

The Hopfield network stores prototypes as energy minima of a recurrent (feedback) network.

Weight matrix (standard bipolar rule: sum of outer products, diagonal zeroed):

$$W = p_1 p_1^T + p_2 p_2^T \quad \text{then set diagonal to zero}$$

Compute $p_1 p_1^T$**:**

$$p_1 p_1^T = \begin{bmatrix} +1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} +1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} +1 & -1 & -1 \\ -1 & +1 & +1 \\ -1 & +1 & +1 \end{bmatrix}$$

Compute $p_2 p_2^T$**:**

$$p_2 p_2^T = \begin{bmatrix} +1 \\ +1 \\ -1 \end{bmatrix} \begin{bmatrix} +1 & +1 & -1 \end{bmatrix} = \begin{bmatrix} +1 & +1 & -1 \\ +1 & +1 & -1 \\ -1 & -1 & +1 \end{bmatrix}$$

Sum, then zero the diagonal:

$$W = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 2 & 0 \\ -2 & 0 & 2 \end{bmatrix} \xrightarrow{\text{zero diagonal}} \boxed{W = \begin{bmatrix} 0 & 0 & -2 \\ 0 & 0 & 0 \\ -2 & 0 & 0 \end{bmatrix}}$$

Bias $\mathbf{b} = \mathbf{0}$. Recall rule: $\mathbf{a}^{\text{new}} = \text{sign}(W\mathbf{a})$; if $n_i = 0$, neuron i keeps its current state.

Recall test – $p_1 = [+1, -1, -1]^T$ **(Orange):**

$$n = W p_1 = \begin{bmatrix} 0 & 0 & -2 \\ 0 & 0 & 0 \\ -2 & 0 & 0 \end{bmatrix} \begin{bmatrix} +1 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} (0)(1) + (0)(-1) + (-2)(-1) \\ (0)(1) + (0)(-1) + (0)(-1) \\ (-2)(1) + (0)(-1) + (0)(-1) \end{bmatrix} = \begin{bmatrix} +2 \\ 0 \\ -2 \end{bmatrix}$$

$$a = \text{sign}\begin{bmatrix}+2\\0\\-2\end{bmatrix} = \begin{bmatrix}+1\\\text{unchanged} = -1\\-1\end{bmatrix} = \begin{bmatrix}+1\\-1\\-1\end{bmatrix} = p_1 \checkmark \quad \textbf{Orange}$$

(Neuron 2 has $n_2 = 0$, so it keeps its input state -1.)

Recall test – $p_2 = [+1, +1, -1]^T$ **(Apple):**

$$n = Wp_2 = \begin{bmatrix}0 & 0 & -2\\0 & 0 & 0\\-2 & 0 & 0\end{bmatrix}\begin{bmatrix}+1\\+1\\-1\end{bmatrix} = \begin{bmatrix}(0)(1)+(0)(1)+(-2)(-1)\\0\\(-2)(1)\end{bmatrix} = \begin{bmatrix}+2\\0\\-2\end{bmatrix}$$

$$a = \text{sign}\begin{bmatrix}+2\\0\\-2\end{bmatrix} = \begin{bmatrix}+1\\\text{unchanged} = +1\\-1\end{bmatrix} = \begin{bmatrix}+1\\+1\\-1\end{bmatrix} = p_2 \checkmark \quad \textbf{Apple}$$

Both prototypes are correctly recalled. The key is neuron 2 (texture): since $w_{2j} = 0$ for all j, it always keeps its input state, acting as a pass-through for texture information.

(e) Testing with Different Inputs and Discussion

Test results for several input patterns:

Input p	**Description**	**Perc.**	**Ham.**	**Hopf.**
$[+1, -1, -1]^T$	Round/rough/light (exact orange)	Orange ✓	Orange ✓	Orange ✓
$[+1, +1, -1]^T$	Round/smooth/light (exact apple)	Apple ✓	Apple ✓	Apple ✓
$[-1, -1, -1]^T$	Ellip./rough/light (1 bit noise from p_1)	Orange	Orange	Spurious
$[+1, +1, +1]^T$	Round/smooth/heavy (1 bit noise from p_2)	Apple	Apple	Spurious
$[-1, +1, -1]^T$	Ellip./smooth/light (1 bit noise from p_2)	Apple	Apple	Spurious
$[+1, -1, +1]^T$	Round/rough/heavy (1 bit noise from p_1)	Orange	Orange	Spurious

Why Hopfield gives spurious attractors for noisy inputs: The two prototypes differ in only *one bit* (texture). With $n = 3$ neurons and only 1 bit of difference between stored patterns, the Hopfield network has very little "basin of attraction" for each pattern. Any single-bit perturbation can push the state outside the correct basin.

Advantages and Disadvantages Summary:

Network	Advantages	Disadvantages
Perceptron	Extremely simple – one weight vector and bias. Fast $O(R)$ classification. Correct on both prototypes. Interpretable: $w_2 = -2$ directly tells us "rough texture = orange."	Hard decision boundary – a single corrupted bit (e.g. texture sensor noise) may give a wrong answer. No confidence measure. Cannot rank similarity.
Hamming	Finds the prototype with minimum Hamming distance. Provides similarity scores for all classes (not just a hard decision). Handles 1-bit sensor noise correctly. Best suited for bipolar pattern classification.	Only works with bipolar/binary inputs. Two-layer architecture is slightly more complex to build. For inputs equidistant from two prototypes, both neurons tie and no winner emerges.
Hopfield	Associative (content-addressable) memory – can recall a full pattern from a partial input. Iterative updates allow multi-step error correction in theory. No separate training phase needed.	Fails on noisy inputs here because p_1 and p_2 differ in only 1 bit (Hamming distance = 1), leaving almost no basin of attraction. Limited storage capacity: $\approx 0.15n$ patterns reliably. Spurious attractors (states that are not stored patterns) may appear.

Conclusion: For this orange-vs-apple problem, the **Hamming network** is the best choice. It correctly classifies both prototypes and provides explicit similarity scores, making it robust to 1-bit sensor noise (e.g. shape or weight misread). The perceptron is also correct but gives no confidence measure. The Hopfield network correctly recalls exact prototypes but is unreliable for noisy inputs because the two stored patterns are too similar (Hamming distance = 1).